

Documento de Diseño de Juego(GDD)

Título del proyecto:

Microjuego_nave

Gestor

Unity

Asignatura:

Fundamentos de Videojuegos

Creado por:

Alba Hernandez Morlesin

1 Escenas

El microjuego consta de una unica escena que acoje todos los elementos del juego incluyiendo el panel de pantalla de pausa la interfaz de usuario (HUD), las mecánicas principales y los elementos del entorno.

2 Assets y GameObjects

Entre los GameObjects podemos distinguir:

- **Ship_0**- Nave jugable controlada por el usuario.
- **Main Camara**- Camara del Juego
- **Space_0**- Fondo del juego
- **Gun**- Punto Spawner para instanciar las balas(Hijo de Player)
- **Bullet_0**- Prefab de bala reutilizable(mediante pooling) que avanza en linea recta tomando como origen la direccion de “Gun” a la hora de disparar
- **Prefab_0**- Prefab de Enemigo base que se desplaza hacia el centro del mapa
- **EnemySpawner**- Spawner encargado de instanciar los asteroides en el perímetro.
- **Canvas**- Encargado de parte grafica y funcional del menu de pausa
- **Poolbalas y PoolMeteor**- GameObjects vacios asociados a un scrip para gestionar los pools de las balas y los enemigos

3 Mecanicas

Para el movimiento avanzar/retroceder del jugador hemos modificado el componente rigidbody mientras que tanto para las balas como la rotacion del jugador lo hemos implementado a traves del componente transform.

Para los enemigos base (meteoros), se ha implementado un sistema de *spawning* basado en una circunferencia imaginaria de radio 11 centrada en el origen de coordenadas (0,0,0). A diferencia de un mapeo aleatorio a lo largo

del eje X fuera de la pantalla impulsado por gravedad, los meteoros se materializan en el perímetro de dicho círculo y se desplazan con una trayectoria vectorial orientada hacia el centro de la escena.

Por otro lado, he implementado un menu de pausa que se acciona tanto con los botones de teclado “esc” y “enter” como físicamente pulsando con el raton encima del pequeño boton de pausa situado en el la esquina superior izq

Con respecto a creacion/destruccion de balas/enemigos he optado por crear dos scripts independientes para crear una funcion de pooling que empieza con 10 instancias, siendo posible aumentar el numero de estas durante el gameplay de ser necesario, con intencion de optimizar la gestion de los recursos. Cada vez que una bala/enemigo colisiona/termina su tiempo de vida, la instancia se desactiva en vez de destruirse, de igual forma a la hora de crear nuevos elementos de este tipo simplemente se activa el primer objeto disponible de la lista.

Tambien se ha incluido un pequeño apartado en el Scrip *Bullet.cs* para que cuando una bala colisione con un enemigo, la escala de este se divide por la mitad, se crea otro minimeteoro y ambos se separan siguiendo la bisetriz de la bala ($\text{dirAsteroide} * \text{dirBala}$).normalized; si por algun casual era 0 utilizo la direccion del asteroide.

Por ultimo, cabe recalcar que todas las colisiones son a su vez gestionadas como triggers ya que con OnCollisionEnter no podia gestionar en caso de que los asteroides se chocaran unos con otros y, al no ser relevantes las fisicas del choque sino capturar el choque en si, he decidido gestionarlasy como atravesamientos

4 Scripts

En el miniproyecto cuento con 7 scrips:

- **Asteroid**-Mantiene las propiedades del asteroide (como su tamaño y velocidad) y se encarga de desactivarse automáticamente pasados 5 segundos.Asociado a prefab Meteor_0.
- **Bullet**-Controla el movimiento de la bala, su desactivación por tiempo y la colisión con los enemigos. Al impactar con un asteroide grande, incrementa la puntuación y lo divide en dos miniasteroides (usando el pool) en un ángulo

específico; si el asteroide ya era pequeño, lo destruye. Asociado a prefab Bullet_0.

- **EnemySpawner**-Genera asteroides de forma progresiva alrededor del jugador en un radio circular, apuntándolos hacia el centro de la pantalla. Aumenta la frecuencia de aparición con el tiempo. Asociado a Main Camara.
- **Pausar**-Gestiona el menú de pausa al pulsar Escape o Enter (deteniendo el tiempo del juego con Time.timeScale), además de permitir reiniciar la escena actual o salir de la aplicación. Asociado a Panel de Pausa y Boton de Pausa .
- **Player**-Controla el movimiento físico de la nave (propulsión y rotación), el disparo de proyectiles (tomándolos de PoolBalas), los límites de la pantalla estilo infinito y la reinicialización de la partida al chocar contra un enemigo. Asociado a Ship.
- **PoolBalas**-Implementa el patrón Object Pooling para instanciar, reutilizar y desactivar balas, evitando instanciar y destruir objetos constantemente. Asociado a GameObject vacio.
- **PoolMeteor**-Iguañ que PoolBalas, pero para los enemigos(Asteroides) Asociado a GameObject vacio.

5 Interaccion

El flujo principal de interacción ocurre cuando una bala colisiona con un asteroide: el proyectil se desactiva instantáneamente, suma un punto a la puntuación global de la interfaz y evalúa el tamaño del enemigo; si se trata de un asteroide grande, este no se destruye sino que se fragmenta en dos miniasteroides reducidos a la mitad y que salen despedidos según a bisectriz formada por las trayectorias de la bala y del impacto original, mientras que si el proyectil impacta a un miniasteroide, este se elimina definitivamente de la escena, y por el contrario, si un asteroide colisiona contra la nave, la puntuación se reinicia a cero y la escena se recarga por completo.